Playable but Private: In-Encoder Selective Encryption with Ascon-AEAD128 and Attack-Adaptive Protection for MJPEG Video on an FPGA SoC

Aravind R, Suhas V J, Rishab H R, Samarth B N, Vijaysurya B S, Chetana G [TODO: order]
Department of Electronics and Communication Engineering, RV College of Engineering, Bengaluru, India
[TODO: e-mails]

***Abstract*—Small video cameras increasingly stream to networks they do not control, yet encrypting the whole stream makes it unplayable without the key and spends energy on every byte. We present a JPEG/MJPEG encoder for a Zynq-7020 FPGA SoC that encrypts only the sign and amplitude bits of selected DCT coefficients, at the entropy-coding stage and with a keystream from the NIST SP 800-232 lightweight standard Ascon-AEAD128. Because Huffman code lengths are untouched, every protection level remains a valid JPEG stream that common decoders open, while a 128-bit tag computed over the frame metadata and the coded scan lets the receiver reject any modified, replayed or forged frame. The register-transfer-level design is verified bit-exact against a Python golden model and needs no additional clock cycles for encryption in any mode. Around the encoder we build an authenticated feedback channel, a key ratchet and an adaptive policy that raises protection when the receiver reports attacks. In a networked simulation, all 11 active attacks we model were mitigated in 0.91 s on average with no false alarms, and a keyed block shuffle removed the outline leakage that sign and amplitude encryption alone leave behind, also against a jigsaw solver and a trained neural attacker. We also report a negative result: a learning bandit policy saves 14 % of encrypted bits but reacts more slowly than a fixed threshold rule.**

***Index Terms*—selective encryption, MJPEG, Ascon, lightweight cryptography, FPGA, authenticated encryption, video surveillance**

I. Introduction

Cameras on doors, corridors and machines now send video over Wi-Fi and shared Ethernet as a matter of course. Two properties matter for such a link. Confidentiality keeps a bystander on the network from watching the scene; integrity lets the viewer trust that what it shows was captured by the camera and not edited, replayed or injected on the way. The usual answer, encrypting the transport with TLS, SRTP [1] or a VPN, provides both, but it hides the stream from every tool that is not part of the key infrastructure and costs cipher work proportional to the full bitrate.

Selective encryption takes a different route: it encrypts only the parts of the compressed representation that carry most of the visual information, and leaves the format intact so that standard decoders still parse the result [2]. For JPEG, the sign and appended amplitude bits of the DCT coefficients can be encrypted without changing any Huffman code length, so the bitstream stays compliant and its size does not grow [3], [4]. The price is that such schemes are easy to weaken: the positions of non-zero coefficients stay visible, and an attacker who guesses the hidden values can still see outlines [2], [5].

This work starts from a practical question: how much of a secure camera link can be put into the encoder hardware itself, and what does an eavesdropper still learn? We make four contributions.

1) An MJPEG encoder in synthesizable Verilog in which an Ascon-AEAD128 [6], [7] keystream is XORed onto coefficient signs and amplitudes as they are entropy-coded, with no added cycles, and a per-frame tag that binds the coded scan to the protection level, session, frame counter and image size (Section III).
2) Six protection levels, from tag-only to a keyed shuffle of macroblocks, evaluated against three keyless attackers (value replacement, a jigsaw solver and a trained U-Net) on a synthetic scene and 4 real video clips (Section V).
3) An authenticated feedback loop with a forward-secure key ratchet and an adaptive policy, evaluated against 11 active attacks and benign impairments; a threshold rule is compared with a contextual bandit over 10 paired seeds (Section VI).
4) A verification flow in which a Python golden model is the reference for every stage: Ascon matches all official known-answer tests, and every RTL block matches the model bit for bit.

Board measurements on the PYNQ-Z2 are in progress; this draft reports register-transfer-level and system simulation results and marks the hardware numbers that are still estimates.

II. Related Work

Selective encryption of JPEG. Van Droogenbroeck and Benedetti encrypted the appended AC bits of baseline JPEG and observed that the stream remains compliant at constant bitrate [3]. Puech and Rodrigues applied AES to selected DCT coefficients of medical images [8], and Auer et al. encrypted JPEG at bitstream level in real time with an AES-based keystream while preserving length and format [4]. Massoudi

et al. survey the field and argue that visual distortion alone is a poor proxy for security, since replacement and chosen-plaintext attacks recover much more than the scrambled image suggests [2]. Lin et al. show that DCT sign bits can be recovered by optimization [5], which bounds what sign-only encryption can promise. Hirose et al. analyse JPEG encryption that shuffles blocks delimited by restart markers [9]; our strongest level uses the same mechanism with a keyed permutation. Selective encryption has also been extended to H.264/AVC and HEVC [10].

Hardware. Kishore et al. added a lightweight scrambling stage to an FPGA JPEG encoder at about 1 % extra hardware [11], and Bağbaba and Örs combined an FPGA JPEG encoder with TEA encryption of DC and low-frequency coefficients [12]. Neither authenticates the frames. Ascon, now standardized by NIST in SP 800-232 [6], [7], was designed for small hardware [13]; Gladis Kurian and Chen implement Ascon on an Artix-7 with Bloom-filter replay protection for IoT messages [14].

Gap. We found no design that (i) places a standardized lightweight AEAD inside a hardware JPEG encoder at the entropy-coding stage, (ii) keeps every protection level playable while authenticating each frame together with its metadata, and (iii) closes the loop from the receiver back to the encoder so that the protection level follows observed attacks. We also could not find an evaluation of the outline leakage of such schemes against learned attackers on real video.

III. System Design

A. Threat model

The attacker sits on the network path. It may observe all traffic, and it may modify, drop, delay, replay or inject both video and feedback datagrams; it does not hold the keys. A passive eavesdropper sends nothing, so it cannot be detected; against it only the configured minimum protection level helps. Physical and side-channel attacks on the device are out of scope. The assets are the visual content of each frame, the integrity of content and metadata (protection level, session, counter and size), and the availability signal that tells the viewer the camera is alive. [TODO: nonce-uniqueness argument and reboot behaviour (GUIDE.md B3)]

B. Encoder and cipher integration

Fig. 1 shows the datapath of a baseline JPEG encoder [15], [16]: colour conversion with 4:2:0 subsampling, a two-pass integer DCT, a reciprocal-multiplication quantiser, zig-zag ordering with run-length coding, the encryption stage, Huffman coding and a bit packer. The whole datapath is fixed-point and mirrors the golden model operation for operation, so the hardware output can be compared byte by byte.

The keystream generator runs Ascon-AEAD128 on an all-zero message under the frame nonce, one permutation round per clock cycle, and keeps a small buffer of keystream bits ahead of the encryption stage. For each coefficient the encryption stage consumes keystream bits only for the fields the protection level selects. If the sign is selected and the next keystream bit is one, all bits of the coefficient's appended amplitude field are inverted; in JPEG's one's-complement amplitude coding this is exactly a sign change. If amplitudes are selected, the remaining bits are XORed onto the lower bits of the field. The size category, and therefore the Huffman code, is never changed. As a result the encrypted scan has the same length as the plain one, and the keystream generator works in parallel with the rest of the pipeline.

TABLE I
Protection levels. All levels carry the per-frame tag.

Level	Encrypted fields
E0	none (authentication only)
E1	AC sign bits
E2	E1 + DC-difference sign bits (default minimum)
E3	E2 + AC amplitude bits
E4	E3 + DC amplitude bits
E5	E4 + keyed permutation of 16×16 macroblocks

A second Ascon instance computes the tag. Its associated data is a 26-byte prefix (format identifier, version, protection level, 64-bit session identifier, 64-bit frame counter, width and height) followed by every byte of the coded scan. The ARM core adds standard JPEG headers and an APP11 segment that carries the prefix and the tag; ordinary viewers skip this segment. Binding the protection level into the tag matters: in an earlier version of our design the level was not authenticated, and an attacker could downgrade a frame without detection.

The 128-bit nonce concatenates the 64-bit session identifier with the 63-bit frame counter and one purpose bit that separates the keystream from the tag computation. The session identifier holds 48 random bits and the current key epoch.

C. Protection levels

Table I lists the levels. Sign and amplitude encryption cannot hide which coefficients are non-zero, because the Huffman symbols reveal run lengths and size categories. E5 therefore inserts a restart marker after every macroblock and permutes the resulting segments with a Fisher–Yates shuffle driven by Ascon under a separately derived key. Restart markers keep the stream decodable but cost 12–21 % bitrate over E4 in our measurements (320×240 and 640×480), whereas E2 adds only the 38-byte APP11 segment (0.3–0.5 %). E5 is implemented in the golden model; restart-marker support in the RTL is ongoing work.

D. Feedback, keys and adaptive protection

The receiver authenticates every frame, checks the counter against a 64-entry replay window as in SRTP [1], and returns heartbeats every 0.5 s and rate-limited alerts. Feedback messages are themselves Ascon-AEAD128 encrypted under a separate key and protected by their own replay window. The video key is advanced by a one-way ratchet, $K_{e+1} = \mathrm{KDF}(K_e, \text{“ratchet”}, e{+}1)$, so a key captured later does not expose earlier epochs. A dead-man rule treats three seconds

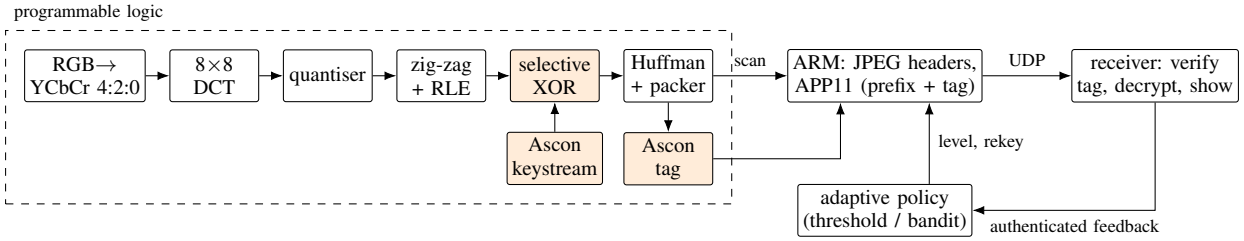


Fig. 1. Encoder datapath with in-loop encryption (shaded) and the feedback loop. The keystream is consumed only for the fields selected by the protection level; the tag covers the frame metadata and the coded scan.

without a valid heartbeat as an attack: the sender switches to the highest level and rekeys.

The sender's ARM core collects evidence in one-second windows: tag failures, replays, malformed and stale frames, frame loss over the last three windows, forged or replayed feedback, and unexpected traffic. Two policies map this evidence to a protection level. The *threshold* policy escalates to the highest level on strong evidence, holds it for 8 s, and rekeys on bursts of tag failures. The *bandit* policy is an ε-greedy contextual bandit [17] over a small discrete context, with optimistic initial values and sample-average updates; its reward penalizes the leakage of the chosen level weighted by the evidence of attack, the fraction of encrypted bits, and level switches. Both policies share the dead-man rule and a minimum rekey interval, and neither can go below the configured minimum level.

IV. Implementation and Verification

The golden model is written in Python and implements Ascon, the integer transforms, entropy coding, encryption and the container format. It passes all 1089 official Ascon-AEAD128 known-answer tests. Each RTL block has a cocotb testbench that drives random inputs with random back-pressure and compares every output with the model; the top level is compared byte for byte for E0–E4 with 64-bit session identifiers and counters above 2^{32}. A self-checking Verilog-2001 testbench, written for Vivado's simulator, encodes five frames and passes in Icarus Verilog. [TODO: XSim result (GUIDE.md 4.1)]

Table II reports encoder cycles per frame. The protection level changes the cycle count by at most 5 cycles per frame, which is noise from handshake timing rather than encryption work: the encoder needs 1.64 cycles per pixel in every mode. At a 100 MHz clock this corresponds to about 199 frames per second at 640×480 and 66 at 1280×720, *provided* that timing closes at 100 MHz. Table III gives the resource use. Until post-route results are available it is a Yosys synthesis estimate: the encoder needs about 5,681 LUTs (10.7 % of the device), and the two Ascon instances account for about 54 % of them. Sharing one Ascon core between keystream and tag is an obvious area optimization that we will evaluate.

TABLE II
Encoder cycles per frame in RTL simulation (160x128, 100 MHz). The last column extrapolates to 640×480 at the same cycles per pixel.

Mode	Cycles	Cycles/pixel	Δ vs E0	fps (640×480)
E0	33515	1.636	+0	199
E1	33517	1.637	+2	199
E2	33513	1.636	-2	199
E3	33517	1.637	+2	199
E4	33510	1.636	-5	199

TABLE III
Resources on the xc7z020 (53 200 LUT, 106 400 FF, 140 BRAM36, 220 DSP). Source: Yosys `synth_xilinx` estimate, **not** post-route; [TODO: replace with Vivado (GUIDE.md 4.1)].

Block	LUT	FF	BRAM36	DSP
Encoder (all)	5681 (10.7%)	2830 (2.7%)	25 (17.9%)	24 (10.9%)
keystream generator	1387	851	0	0
tag unit	1709	1204	0	0

V. Security Evaluation

Setup. We measure what a keyless observer can reconstruct. For every frame we take the better of two pictures: the protected stream decoded as is, and a *replacement attack* that substitutes fixed guesses for the encrypted values (absolute values for encrypted signs, the smallest value of the size category for encrypted amplitudes, zero DC differences) and re-encodes a valid JPEG. Leakage is the Pearson correlation between the Sobel gradient magnitudes of this picture and of the original (edge correlation); we also report SSIM [18]. Sources are a synthetic corridor scene and 4 publicly available HD clips (two film trailers with fast cuts and two talking-head videos, 1280 pixels wide), resized to 320×240 and sampled every 150 frames. Frames without edges, such as black transition frames, are excluded from the correlation.

Format compliance. The protected streams of all six levels decode without error in Pillow, OpenCV, scikit-image, FFmpeg (MJPEG), at the original resolution.

Outline leakage. Fig. 2 and Table IV show the main effect. Under E1–E4 the eavesdropper sees a clear outline of the scene: edge correlation stays between 0.27 and 0.58 on the real clips, and encrypting amplitudes in addition to signs (E3, E4) does not reduce it, because the positions and size categories of the non-zero coefficients remain public. The keyed macroblock permutation of E5 removes the outline (edge correlation between -0.004 and 0.017). Fig. 3 plots the

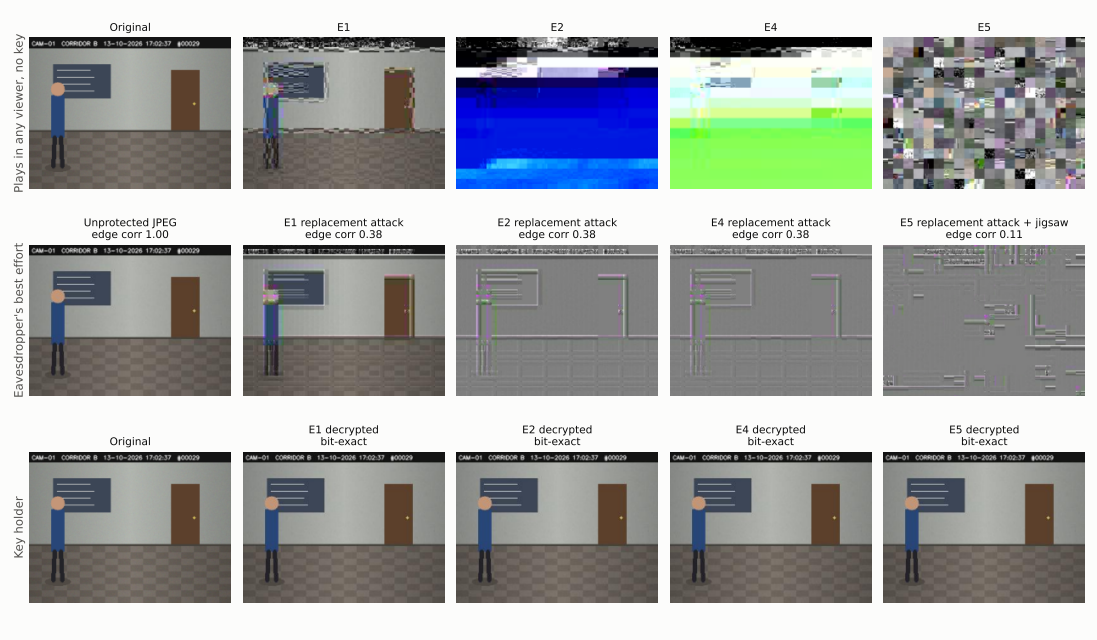


Fig. 2. One synthetic frame per protection level. Top: what any viewer shows without the key. Middle: the eavesdropper's best reconstruction (replacement attack; for E5 followed by the jigsaw solver) with its edge correlation. Bottom: the key holder's output, bit-exact with the unprotected encoder. [TODO: replace with an openly licensed real frame (GUIDE.md B2)]

TABLE IV
OUTLINE LEAKAGE: EDGE CORRELATION BETWEEN THE ORIGINAL AND THE EAVESDROPPER'S BEST KEYLESS RECONSTRUCTION (DECODE OR REPLACEMENT ATTACK), 20 FRAMES PER SOURCE (0 = NO OUTLINE). "ENC." IS THE FRACTION OF PAYLOAD BITS ENCRYPTED.

Mode	Enc.	Synthetic	Trailer A	Trailer B	Talk A	Talk B
E0	0.000	1.00	0.99	0.99	0.99	0.99
E1	0.114	0.38	0.47	0.58	0.50	0.49
E2	0.134	0.36	0.34	0.27	0.44	0.38
E3	0.258	0.37	0.34	0.31	0.44	0.44
E4	0.295	0.37	0.33	0.30	0.44	0.43
E5	0.310	0.00	0.01	0.02	-0.00	0.01

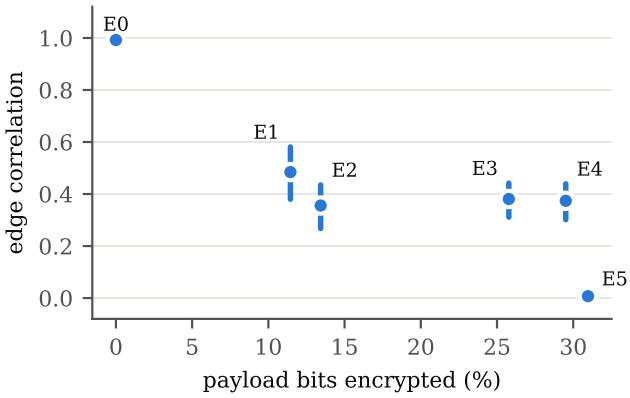


Fig. 3. Outline leakage against encrypted share per level. Dots: mean over the five sources; bars: range.

same data against the share of encrypted payload bits: E2 encrypts 13.4 % of the bits, E4 29.5 %, and E5 only slightly more, 31.0 %, yet only E5 moves leakage to zero.

Jigsaw attacker. A permutation of blocks invites a puzzle solver. We implemented a greedy region-growing solver in the style of Pomeranz et al. [19], with gradient-continuation compatibility normalized by the second-best match. On unencrypted shuffled tiles it restores most neighbour relations, which confirms that it works. On E5 replacement-attack tiles it places at most 1.7 % of neighbour pairs correctly, and the solved picture has no outline (Table V). The structure is not destroyed, only hidden: an oracle that knew the permutation would recover edge correlations of 0.41–0.47. A stronger solver could do better than ours, so this result is a lower bound on what E5 can leak, not a proof that it leaks nothing.

Learned attacker. We trained a U-Net [20] to map the two keyless pictures (decoded and replacement attack) to the original, on 1000 frames of one clip, and tested it on 400 frames of a different clip, for 25 epochs (Table VI). The network learns the overall brightness and colour layout and raises SSIM for E2 and E4, strongly in the trailer pair (up to 0.43), but it does not create an outline under E5: the edge correlation is at most 0.021 in absolute value. We therefore claim that E5 hides the shape of the scene from these attackers, not that it hides everything.

VI. PROTOCOL AND DEFENCE EVALUATION

Setup. Sender, network channel, receiver and logger run as separate processes that talk over UDP, so the evaluation exercises the real wire format, fragmentation and timing. The

TABLE V
E5 AGAINST A GREEDY JIGSAW SOLVER (8 FRAMES PER SOURCE). "ORACLE" PLACES EVERY BLOCK CORRECTLY, WHICH IS THE OUTLINE E5 HIDES.

Source	Correct neighbours (%)	Edge corr. solved	Edge corr. oracle
Synthetic	2.3	0.059	0.374
Trailer A	1.4	-0.019	0.460
Trailer B	1.3	-0.011	0.413
Talk A	1.4	0.005	0.449
Talk B	1.7	-0.000	0.467

TABLE VI
KEYLESS RECONSTRUCTION BY A U-NET TRAINED ON ONE CLIP AND TESTED ON ANOTHER (P1: TRAILER A→B, P2: TALK A→B). ENTRIES: PSNR (DB) / SSIM / EDGE CORRELATION.

Pair	Mode	Replacement attack	Learned attack
P1	E1	20.8 / 0.53 / 0.39	24.9 / 0.72 / 0.53
P1	E2	8.0 / 0.09 / 0.41	19.1 / 0.42 / 0.33
P1	E4	8.0 / 0.10 / 0.38	19.1 / 0.43 / 0.34
P1	E5	8.1 / 0.08 / -0.01	17.3 / 0.34 / -0.02
P2	E1	18.5 / 0.48 / 0.41	22.9 / 0.70 / 0.66
P2	E2	10.7 / 0.20 / 0.38	11.9 / 0.27 / 0.34
P2	E4	10.8 / 0.22 / 0.37	12.4 / 0.28 / 0.32
P2	E5	10.9 / 0.16 / 0.01	12.8 / 0.26 / 0.02

TABLE VII
REACTION TO EACH ACTIVE ATTACK (FULL SUITE, THRESHOLD POLICY, 3 RUNS; MEAN ± S.D. IN SECONDS). "FIRST REJECTION" IS WHEN THE RECEIVER FIRST DISCARDS A FRAME; "MITIGATED" IS WHEN THE SENDER HAS RAISED PROTECTION OR REKEYED.

Attack	Mitigated	First rejection	Mitigated after
bit flips in the scan	3/3	0.09 ± 0.03	0.41 ± 0.02
mode downgrade in header	3/3	0.06 ± 0.02	0.44 ± 0.02
splice of two frames	3/3	0.13 ± 0.00	0.46 ± 0.01
forged frame injection	3/3	0.14 ± 0.06	0.44 ± 0.03
frame replay	3/3	0.07 ± 0.05	0.46 ± 0.03
frame delay	3/3	4.13 ± 0.04	1.46 ± 0.03
frame dropping	3/3	–	1.46 ± 0.03
junk flood	3/3	–	0.49 ± 0.04
feedback dropping	3/3	–	3.47 ± 0.05
forged feedback	3/3	–	0.46 ± 0.04
feedback replay	3/3	–	0.44 ± 0.01

TABLE VIII
THRESHOLD RULE VS. ε-GREEDY BANDIT ON THE LEARNING SCENARIO, 10 SEEDS EACH (MEAN ± S.D.). THE LAST COLUMN COUNTS THE SEEDS IN WHICH THE BANDIT WAS BETTER (LOWER) THAN THE THRESHOLD RULE WITH THE SAME SEED.

Metric	Threshold	Bandit	Bandit better
Mitigation (s)	0.46 ± 0.02	1.34 ± 0.03	0/10
Leakage under attack	0.000 ± 0.001	0.054 ± 0.004	0/10
Encrypted share	0.188 ± 0.001	0.162 ± 0.001	10/10
Encrypted share, calm	0.135 ± 0.000	0.138 ± 0.001	0/10

sender encodes 320×240 video at 10 frames per second with the golden model, which produces the same bytes as the RTL. The channel can impose loss, delay, jitter and reordering, and it implements the attacker of our threat model. All results below come from a 4-core cloud virtual machine with one scenario running at a time.

Active attacks. The full scenario applies 11 attacks for six seconds each, 25 s apart: bit flips in the scan, a mode downgrade in the header, splicing of two frames, injection of a forged frame, replay, delay, dropping, a junk flood, and dropping, forging and replaying feedback. Table VII gives the result over 3 runs. Every attack was mitigated in every run, on average 0.91 ± 0.02 s after it began, with 0/375 false alarms in calm windows and a median frame latency of 97 ms. Attacks that corrupt frames are visible at the receiver within tenths of a second. The slowest case, dropped feedback, is caught only by the dead-man rule, after about 3.5 s: the 3 s timeout plus part of a decision window. With a trailer and a talking-head clip as the video source instead of the synthetic scene, all attacks were again mitigated (11/11 and 11/11, means 0.89 and 1.09 s) without false alarms.

Benign impairments. A second scenario applies only network problems: rising packet loss up to 6 %, delay up to 120 ms with jitter, and 5 % reordering. Over 3 runs the defence raised no false alarm (0/447 windows); median latency was 82 ms and the 95th percentile 278 ms. Two design choices made this possible: loss is judged over three windows instead of one, and late frames are distinguished from replays by the replay window.

Threshold rule against bandit. The learning scenario repeats a bit-flip and a replay attack five times each, separated by calm periods. We ran both policies with 10 seeds; the same seed drives the bandit's exploration, and the channel adds its own randomness. Table VIII and Fig. 4 show a consistent trade-off rather than a winner. The bandit encrypted fewer bits in every seed (0.162 against 0.188 of the payload, 14 % less), but it reacted later (1.34 against 0.46 s) and leaked more while attacks were active (0.054 against almost zero), again in every seed. Neither policy raised a false alarm (0/6020). For a security link we keep the threshold rule as the default and treat the bandit as an option where bandwidth or energy matters more than the first second of an attack.

Cost relative to software. On the simulation host, our Python model needs 134 ms to encode a 640×480 frame and a further 151 ms to apply E2 selective encryption to the finished JPEG, whereas the optimized libjpeg-turbo encoder takes 0.95 ms and full-stream AES-GCM adds about 0.01 ms. A desktop processor is therefore not the right point of comparison: the case for doing this in the encoder rests on the embedded processor of the same SoC, which has no AES instructions, and on keeping the output playable and authenticated. [TODO: ARM Cortex-A9 baseline (libjpeg-turbo, AES-GCM, selective encryption in C), FPGA frame rate and energy per frame measured on the PYNQ-Z2 (GUIDE.md A1–A4, B1)]

VII. DISCUSSION AND LIMITATIONS

Three limitations bound our claims. First, the hardware numbers above come from RTL simulation and a synthesis

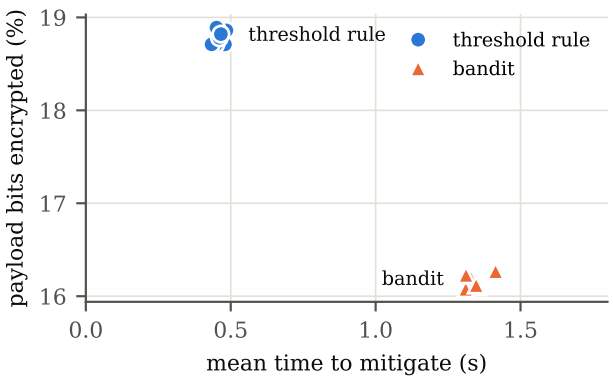


Fig. 4. Each point is one seed of the learning scenario. The bandit (triangles) trades a slower reaction for fewer encrypted bits.

estimate; post-route timing, resources and board measurements are pending, and E5 exists only in the golden model. Second, sign and amplitude encryption leak outlines by construction, and our measurements show that encrypting more bits does not fix this; only a change of block positions does, at a bitrate cost. Third, our attackers are greedy or trained on a single clip; stronger puzzle solvers or attackers trained on large video corpora may extract more from E5. On the protocol side, the receiver cannot tell heavy benign loss from partial jamming, so only near-total suppression escalates, and the reaction time is bounded below by the one-second decision window. The key derivation uses the Ascon-AEAD128 tag as a pseudo-random function; switching to the standard's extendable-output function is planned. The test clips are copyrighted, which is why the paper reports numbers on them but shows only synthetic frames.

VIII. Conclusion

Encrypting coefficient signs and amplitudes with Ascon inside a JPEG encoder costs no cycles, keeps every protection level playable, and, combined with a per-frame tag over metadata and scan, turns a plain MJPEG stream into an authenticated one. Our measurements also show the limit of the idea: without a keyed change of block positions, an eavesdropper still sees the outline of the scene, whatever share of bits is encrypted. A closed feedback loop detects and answers active attacks within about a second in simulation. The next steps are post-route and on-board results on the PYNQ-Z2, macroblock shuffling in hardware, a shared Ascon core, and evaluation on openly licensed video.

Acknowledgment

We thank Dr. Chetana G for her guidance. [TODO: AI-assistance disclosure as required by the conference (see paper/README.md): name the tools and what they were used for, e.g. code generation, test scaffolding and language editing.]

References

[1] M. Baugher, D. McGrew, M. Naslund, E. Carrara, and K. Norrman, "The secure real-time transport protocol (SRTP)," IETF, RFC 3711, Mar. 2004.

[2] A. Massoudi, F. Lefebvre, C. De Vleeschouwer, B. Macq, and J.-J. Quisquater, "Overview on selective encryption of image and video: Challenges and perspectives," *EURASIP Journal on Information Security*, vol. 2008, p. 179290, 2008.

[3] M. Van Droogenbroeck and R. Benedetti, "Techniques for a selective encryption of uncompressed and compressed images," in *Proc. Advanced Concepts for Intelligent Vision Systems (ACIVS)*, Ghent, Belgium, Sep. 2002, pp. 90–97.

[4] S. Auer, A. Bliem, D. Engel, and A. Unterweger, "Bitstream-based JPEG encryption in real-time," *International Journal of Digital Crime and Forensics*, vol. 5, no. 3, pp. 1–14, 2013.

[5] R. Lin, S. Liu, J. Jiang, S. Li, C. Li, and C.-C. J. Kuo, "Recovering sign bits of DCT coefficients in digital images as an optimization problem," *Journal of Visual Communication and Image Representation*, p. 104045, 2024.

[6] M. S. Turan, K. A. McKay, D. Chang, J. Kang, and J. Kelsey, "Ascon-based lightweight cryptography standards for constrained devices: Authenticated encryption, hash, and extendable output functions," National Institute of Standards and Technology, NIST Special Publication 800-232, Aug. 2025.

[7] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "Ascon v1.2: Lightweight authenticated encryption and hashing," *Journal of Cryptology*, vol. 34, no. 3, p. 33, 2021.

[8] W. Puech and J. M. Rodrigues, "Crypto-compression of medical images by selective encryption of DCT," in *Proc. 13th European Signal Processing Conference (EUSIPCO)*, Antalya, Turkey, 2005.

[9] M. Hirose, S. Imaizumi, and H. Kiya, "On the security of bitstream-level JPEG encryption with restart markers," arXiv:2410.06522, 2024.

[10] B. Boyadjis, C. Bergeron, B. Pesquet-Popescu, and F. Dufaux, "Extended selective encryption of H.264/AVC (CABAC)- and HEVC-encoded video streams," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 27, no. 4, 2017.

[11] B. Kishore, B. K. Shreyamsha Kumar, and C. R. Patil, "FPGA based simple and fast JPEG encryptor," *Journal of Real-Time Image Processing*, vol. 10, no. 3, pp. 551–559, 2015.

[12] A. Ç. Bağbaba and B. Örs, "Hardware implementation of novel image compression-encryption system on a FPGA," in *Proc. 9th International Conference on Electrical and Electronics Engineering (ELECO)*, Bursa, Turkey, 2015, pp. 1159–1163.

[13] H. Groß, E. Wenger, C. Dobraunig, and C. Ehrenhöfer, "Ascon hardware implementations and side-channel evaluation," *Microprocessors and Microsystems*, vol. 52, pp. 470–479, 2017.

[14] M. Gladis Kurian and Y. Chen, "Ascon on FPGA: Post-quantum safe authenticated encryption with replay protection for IoT," *Electronics*, vol. 14, no. 13, p. 2668, 2025.

[15] ITU-T, "Information technology – digital compression and coding of continuous-tone still images – requirements and guidelines," International Telecommunication Union, Recommendation T.81, 1992.

[16] G. K. Wallace, "The JPEG still picture compression standard," *IEEE Transactions on Consumer Electronics*, vol. 38, no. 1, pp. xviii–xxxiv, 1992.

[17] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[18] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[19] D. Pomeranz, M. Shemesh, and O. Ben-Shahar, "A fully automated greedy square jigsaw puzzle solver," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011, pp. 9–16.

[20] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, ser. LNCS, vol. 9351, 2015, pp. 234–241.